POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1

FINAL REPORT

Retail Revenue Analysis BI System

Class: E22HTTT

Name: Nguyễn Đức Anh - B22DCDT017

Nguyễn Thiên Thịnh - B22DCCN835

Mentor: Kim Ngọc Bách

Ha Noi – 2026

TABLE OF CONTENT

I. Problem Introduction

1.1. Background

In consumer goods enterprises, business data is usually generated from many departments such as sales, marketing, finance, production, and operations. If data is scattered across Excel files or separate systems, managers will find it difficult to fully monitor revenue, profit, costs, sales channel performance, and the contribution of each region.

The BI_MASAN_revenue project simulates the development of a BI system for Masan revenue data. The original data from the masan_case.xlsx file is loaded into three source systems: PostgreSQL for sales/CRM, MySQL for finance/marketing, and MongoDB for production/operations. The data is then processed using Spark following the Medallion Architecture, including Bronze, Silver, and Gold layers, before being loaded into the gold schema of the Data Warehouse to support the Power BI dashboard.

1.2. Problems to Solve

The main problem is to transform raw, distributed, and unstandardized data into centralized and reliable analytical data. The key issues include:

• Data is stored in multiple sources: PostgreSQL, MySQL, MongoDB, and Excel files.

• Raw data contains errors such as missing values, incorrect date formats, wrong data types, or duplicates.

• Source systems are designed for operations and are not suitable for BI analytical queries.

• Revenue, profit, cost, marketing, production, and logistics data need to be integrated into a single model.

• Users need a visual dashboard to monitor KPIs and quickly extract insights.

Therefore, the project needs to build an ETL process, a Data Warehouse, and a dashboard to standardize data, aggregate key metrics, and support business decision-making.

1.3. Detailed Objectives

The main objectives of the project are:

• Simulate a multi-source data system including PostgreSQL, MySQL, and MongoDB.

• Load raw data from masan_case.xlsx into the source systems.

• Build a Spark pipeline following the three layers: Bronze, Silver, and Gold.

• Clean data, convert data types, handle null values, and remove invalid data.

• Design the gold Data Warehouse schema using an extended Star Schema model.

• Connect Power BI to the Data Warehouse to build an analytical dashboard.

• Analyze insights by revenue, profit, region, product category, channel, and customer segment.

• Propose business performance improvement actions based on dashboard results.

1.4. Analytical Questions

The dashboard is built to answer the following key questions:

• What are the current total revenue, profit, Profit Margin, and Marketing Cost?

• Which region contributes the best revenue and profit?

• Which product category has high revenue, and which category has a strong profit margin?

• Which sales/advertising channel is the most effective in terms of revenue and profit?

• Which customer segment delivers the highest business performance?

• What is the relationship between ROI and profit?

• Which groups should be expanded, and which groups need cost optimization?

1.5. Project Scope

The project scope focuses on building a simulated BI pipeline from raw data to an analytical dashboard. The dataset used is masan_case.xlsx, which contains 2,964 rows and 37 columns, including information about orders, products, regions, branches, costs, marketing, production, and market data.

The project implements three source systems, a Data Lake following the Medallion Architecture, a PostgreSQL Data Warehouse with the gold schema, a Spark pipeline, and a Power BI dashboard. The report focuses on analyzing revenue, profit, marketing cost, product categories, regions, sales channels, and customer segments.

Topics such as real-time ETL, production-level security, machine learning, advanced forecasting, or performance evaluation on large-scale real data are outside the main scope of this project.

II. Dataset Description

2.1. Data Overview

The dataset used in this project is the Excel file masan_case.xlsx, which serves as the initial raw data source for the entire BI system. This file simulates business data of a consumer goods company, where each row represents a record related to sales activities, products, business regions, sales channels, marketing campaigns, costs, profits, and several operational details. The dataset does not only contain revenue data, but also combines various groups of information to support multidimensional analysis.

In this project, the dataset plays two roles. First, it is the input data loaded into source systems such as PostgreSQL, MongoDB, and MySQL. Second, it is the basis for designing dimension and fact tables in the Data Warehouse, from which the Power BI dashboard is built to support revenue and business performance analysis. Therefore, understanding the dataset structure is an important step before designing the ETL process and the analytical data model.

General information of the dataset:

Attribute	Value
Source file	masan_case.xlsx
Number of rows	2,964
Number of columns	37
Number of valid parsed dates	2,851
Number of unique OrderIDs	2,959
Number of duplicate OrderID rows	5
Number of regions	3
Number of branches	9
Number of departments	6

Attribute	Value
Number of products	8
Number of categories	4
Number of sales/marketing channels	6
Number of customer segments	4
Number of promotion campaigns	6

The dataset contains 37 columns, covering many different aspects of business operations. Some columns are identifier data such as OrderID, Product, Branch, and Region; some columns are time-related data such as Date, Year, and Month; and some columns are measurement data such as Revenue, Cost, Profit, Budget, MarketingCost, and LogisticsCost. In addition, the dataset also includes pre-calculated indicators such as ProfitMargin, ROI_Marketing, ConversionProxy, MarketShare, and RegionProfitShare.

The data period ranges from January 2, 2023 to May 31, 2026 for rows with valid parsed dates. Since the data is simulated, some rows may contain invalid dates or data after the time of writing the report. This point should be clearly stated in the report to avoid misunderstanding the dataset as real operational data.

2.2. Value Scope of Analytical Dimensions

The main analytical dimensions in the dataset include region, branch, department, product, category, sales channel, customer segment, and marketing campaign. These dimensions are used to create slicers, filters, and analytical charts in the dashboard.

Data dimension	Values in the dataset
Region	North, South, Central
Branch	Binh Duong, Can Tho, Hue, Hanoi, Hai Phong, Nha Trang, Quang Ninh, Ho Chi Minh City, Da Nang
Department	Distribution, Finance, Marketing, Operations, Other, Raw milk

Category	Ice cream, Powdered milk, Yogurt, Fresh milk
Product	Boxed ice cream, Ice cream bar, Dielac powdered milk, Optinum powdered milk, Unsweetened yogurt, Spoonable yogurt, Unsweetened fresh milk, Very low-sugar fresh milk
Stage	Add to cart, Click, Impression, Purchase, Visit website
Channel	Ecommerce, Facebook, Other, Retail promotion, TV, Traditional trade
CustomerSegment	Retail, Modern trade channel, Distributor, E-commerce
PromotionCampaign	Back-to-School, Mid-Autumn Campaign, Spring Campaign, Summer Promotion, Tet Promotion, Year-End Promotion
Machine	A, B, C, D

The above values show that the dataset is designed to support business analysis at multiple levels. Users can start from an overall company-wide perspective, then drill down into each region, branch, product group, or campaign. This is the basis for building a Star Schema model, in which qualitative columns such as Region, Branch, Product, Category, Channel, CustomerSegment, and PromotionCampaign are placed into dimension tables.

2.3. Main Data Field Groups

The dataset can be divided into several groups of fields based on business meaning. This grouping helps identify which columns belong to dimensions, which columns belong to facts, and which columns need to be processed during the ETL process.

Data group	Related columns	Meaning
Orders	OrderID, Date, Year, Month	Transaction information over time
Geography	Region, Branch	Region and branch
Products	Product, Category	Product and category

Production	Department, Stage, Machine, InventoryLevel, RawMaterialCost, LaborCost	Operational/production information
Sales	Channel, CustomerSegment, Quantity, Revenue, RevenuePerUnit	Revenue by channel and segment
Costs	Cost, UnitCost, LogisticsCost, MarketingCost	Different types of costs
Finance	Profit, ProfitMargin, Budget, Target, MarketSize, MarketShare	Business performance indicators
Marketing	PromotionCampaign, ROI_Marketing, ConversionProxy	Campaigns and marketing effectiveness
Customer service	Calls, WaitingTime, Information	Data supporting service analysis

Overall, the dataset is designed by combining many types of data into a single Excel table. This organization is convenient for providing the initial input data, but it is not optimal for large-scale BI analysis. Therefore, in this project, the data needs to be separated into different source systems and then integrated into the Data Warehouse using an analytical model.

2.4. Detailed Data Dictionary

This section describes the meaning of each column in the dataset. It serves as the basis for identifying ETL processing logic, designing source tables, and designing the Data Warehouse.

2.4.1. Order Identifier and Time Group

Column	Observed data type	Meaning	Analytical role
OrderID	Integer	Identifier of the order or transaction	Used to identify transactions, check duplicates, and serve as a key in the fact table

Date	String/date	Date when the transaction occurred	Used for analysis by day, month, quarter, and year
Year	Integer	Year of the transaction	Supports yearly aggregation
Month	Integer	Month of the transaction	Supports monthly aggregation

During the ETL process, Date is an important column but also one that requires careful processing because it contains missing values or values that cannot be parsed into valid dates. After cleaning, the date information is used to create the dim_date table in the Data Warehouse.

2.4.2. Geography and Organization Group

Column	Observed data type	Meaning	Analytical role
Region	String	Business region	Analyze revenue, profit, and target by region
Branch	String	Sales branch or business location	Analyze the performance of each branch
Departme nt	String	Related department or division	Support operational and internal cost analysis

This group of columns helps analyze differences in business performance across regions. For example, the dashboard can answer which region has the highest revenue, which branch has the best profit, or which area needs cost optimization.

2.4.3. Product and Category Group

Column	Observed data type	Meaning	Analytical role
Product	String	Product name	Analyze revenue, quantity, and profit by product

Category	String	Product category	Analyze revenue structure by category

The Product and Category columns are the basis for creating the product dimension. In the dashboard, users can compare revenue among categories such as powdered milk, yogurt, fresh milk, and ice cream. In addition, it is possible to identify products with high revenue but low profit in order to propose adjustments to pricing, costs, or sales strategies.

2.4.4. Sales and Customer Group

Column	Observed data type	Meaning	Analytical role
Channel	String	Sales channel or marketing channel	Analyze the effectiveness of each channel
CustomerSegment	String	Customer segment	Analyze revenue and profit by customer group
Quantity	Float	Quantity sold	Calculate sales volume and support revenue-per-unit calculation
Revenue	Float	Revenue of the transaction	Main measure in the fact table
RevenuePerUnit	Float	Revenue per product unit	Analyze average sales value

This group of columns supports questions such as which sales channel contributes the most revenue, which customer segment generates the best profit, or whether sales volume correlates with revenue. Revenue is the most important measure in the problem and is used in most dashboard charts.

2.4.5. Cost and Profit Group

Column	Observed data type	Meaning	Analytical role
Cost	Float	Total cost related to the transaction	Calculate profit and cost ratio

Profit	Float	Profit	Main measure for evaluating business performance
ProfitMargin	Float	Profit margin	Evaluate the quality of revenue
UnitCost	Float	Cost per unit	Analyze cost efficiency by product
RawMaterialCost	Float	Raw material cost	Analyze production costs
LaborCost	Float	Labor cost	Analyze operating costs
LogisticsCost	Float	Logistics cost	Evaluate the impact of transportation/warehousing
MarketingCost	Float	Marketing cost	Evaluate marketing investment effectiveness
CostPerMachine Unit	Float	Cost per machine/production unit	Support operational analysis

The cost group is important for analyzing business performance. High revenue does not necessarily mean good performance if costs are also high. Therefore, the dashboard needs to combine revenue with costs and profit to provide more accurate insights.

2.4.6. Finance, Budget, and Market Group

Column	Observed data type	Meaning	Analytical role
Budget	Float	Allocated budget	Compare revenue/costs with budget
Target	Float	Revenue target or business target	Evaluate target achievement
MarketSize	Float	Market size	Evaluate market potential

MarketShare	Float	Market share	Analyze market position
RegionProfitShare	Float	Profit share by region	Compare profit contribution among regions

This group of columns supports management-level analysis, especially comparisons between actual performance and plans. For example, users can check which region meets targets well, which region uses its budget inefficiently, or which market still has growth potential.

2.4.7. Marketing, Behavior, and Customer Service Group

Column	Observed data type	Meaning	Analytical role
PromotionCampaign	String	Name of the promotion/marketing campaign	Analyze the effectiveness of each campaign
ROI_Marketing	Float	Marketing return-on-investment indicator	Evaluate marketing cost efficiency
ConversionProxy	Float	Proxy indicator for conversion	Evaluate marketing funnel effectiveness
Stage	String	Stage in the customer journey	Analyze behavior or funnel
Calls	Float	Number of calls or contacts	Analyze customer service activities
WaitingTime	Float	Waiting time	Evaluate service quality
Information	String	Additional information	Can be used to describe status or business notes

This group of columns extends the analysis from revenue to marketing effectiveness and customer experience. Within the scope of the revenue dashboard, columns such as PromotionCampaign, MarketingCost, and ROI_Marketing play the most important roles.

2.4.8. Production and Operations Group

Column	Observed data type	Meaning	Analytical role
Machine	String	Machine or production line code	Analyze operations by machine
InventoryLevel	Float	Inventory level	Support evaluation of the relationship between inventory and sales
Stage	String	Operational or behavioral stage	Can be used for funnel or operational analysis
Department	String	Responsible department	Analyze costs by department

Operational columns are not the only focus of the revenue dashboard, but they provide additional context when analyzing costs and business efficiency. For example, low profit may be related to raw material costs, labor costs, inventory, or logistics costs.

2.5. Data Distribution into Source Systems

After understanding the dataset structure, the project divides the data from the Excel file into three source systems. This distribution simulates how data is managed in an enterprise, where each department uses a separate system for its own business operations.

PostgreSQL — Sales/CRM

PostgreSQL is used to store sales and CRM data. This is a relational data source, suitable for tables with clear structures such as orders, order details, products, and branches.

Tables initialized in init-postgres.sql:

Table	Purpose
categories	Product category catalog
products	Product catalog
branches	Branches and regions
orders	Order information
order_details	Order details by product

Data loading script: load_data/load_postgres.py.

In this script, data is read from Excel, column names are standardized, and the data is split into smaller tables. For example, the product catalog is loaded into products, branches are loaded into branches, orders are loaded into orders, and indicators such as quantity, unit price, and unit cost are loaded into order_details. This separation helps simulate a sales system with primary keys, foreign keys, and reduced duplication of master data.

MongoDB — Production/Operations

MongoDB is used to store production and operational data. This is a NoSQL source, suitable for simulating operational data with flexible structures such as production logs, machines, departments, inventory, and logistics costs.

Collections initialized in init-mongo.js:

Collection	Purpose
Categories	Product category catalog
Products	Product catalog
Departments	Departments/factories

ProductionLogs	Production logs
LogisticsCosts	Logistics costs by date/branch

Data loading script: load_data/load_mongo.py.

In this script, product data, category data, department data, production logs, and logistics costs are loaded into the corresponding collections. Fields such as InventoryLevel, RawMaterialCost, LaborCost, and LogisticsCost help extend the analysis from revenue to operations and costs. This data is useful when explaining the causes of low profit or increasing costs.

MySQL — Finance/Marketing

MySQL is used to store finance and marketing data. This data group supports the evaluation of budgets, targets, marketing costs, and campaign effectiveness.

Tables initialized in init-mysql.sql:

Table	Purpose
marketing_campaigns	Marketing campaign catalog
daily_marketing_spend	Daily marketing spend by date/region
monthly_budgets	Budget, target, and market size by month/region

Data loading script: load_data/load_mysql.py.

In this script, marketing campaigns are loaded into marketing_campaigns, daily marketing costs are loaded into daily_marketing_spend, and budget, target, and market size by month/region are loaded into monthly_budgets. This data group helps the dashboard answer questions such as which campaign is effective, which region meets its target, and whether marketing costs are proportional to revenue.

2.6. Role of the Dataset in BI Analysis

The masan_case.xlsx dataset plays a central role in the entire project because it provides identifier data, transaction data, cost data, and KPI data. From this dataset, the system can build many different analytical perspectives:

- Revenue analysis: using Revenue, Quantity, and RevenuePerUnit combined with Date, Region, Branch, Product, Category, and Channel.
- Profit analysis: using Profit, Cost, ProfitMargin, UnitCost, RawMaterialCost, and LaborCost.
- Marketing analysis: using PromotionCampaign, MarketingCost, ROI_Marketing, ConversionProxy, and Channel.
- Budget analysis: using Budget, Target, MarketSize, and MarketShare.
- Operational analysis: using Department, Machine, InventoryLevel, LogisticsCost, and CostPerMachineUnit.
- Customer analysis: using CustomerSegment, Calls, WaitingTime, and Information.

When loaded into the Data Warehouse, qualitative columns are converted into dimensions, while numeric columns are placed into the fact table as measures. As a result, the dashboard can display overall KPIs and allow users to filter data by each analytical dimension. This is why the dataset needs to be described carefully in the report: understanding the dataset is the first condition for designing the ETL process, the Data Warehouse, and the dashboard correctly.

III. Data Warehouse Design

3.1. Role of the Data Warehouse in the Project

The Data Warehouse is the final data layer used for BI analysis. After the data is loaded into the source systems, including PostgreSQL, MySQL, and MongoDB, Spark processes the data through three layers of the Medallion Architecture: Bronze, Silver, and Gold. In this architecture, the Gold layer serves as the Data Warehouse for Power BI. Data in this layer has been cleaned, standardized, enriched through dimension key lookups, and reorganized according to an analytical data model.

The objective of the Data Warehouse is to create a stable, understandable, and optimized data layer for reporting. Power BI users do not need to query PostgreSQL, MySQL, or MongoDB source systems directly. Instead, all important data is consolidated into the Gold layer and organized into clear dimension and fact tables.

3.2. Reason for Choosing an Extended Star Schema

For the Masan revenue BI problem, the data needs to be analyzed across multiple dimensions such as time, product, branch, region, marketing campaign, department, and cost groups. At the same time, the project does not only analyze sales revenue, but also analyzes marketing spend, budget, production logs, and logistics costs. Therefore, the suitable model is not just a single Star Schema, but an extended Star Schema in the form of a Fact Constellation.

A Fact Constellation is a model that contains multiple fact tables sharing common dimension tables. In this project, the gold.dim_date table is shared by most fact tables; gold.dim_product is shared by sales and production data; and gold.dim_branch is shared by sales and logistics data. As a result, the Power BI dashboard can analyze different business domains while maintaining consistent analytical dimensions.

The reasons for choosing this model are as follows:

Clearly separates descriptive data and measurement data: dimension tables store descriptive information such as date, product, branch, department, and campaign, while fact tables store metrics such as revenue, profit, marketing cost, inventory level, and logistics cost.

Supports multiple business domains: Sales, Finance/Marketing, and Production/Logistics have separate fact tables, but they can still be analyzed together through shared dimensions such as date, product, or branch.

Optimized for Power BI: one-to-many relationships from dimensions to facts make the data model easier to filter, easier to aggregate, and help avoid complex many-to-many relationships.

Easy to extend: if future requirements include additional facts such as customer service, inventory snapshots, or campaign conversion, new fact tables can be added without breaking the current model.

Easier to control data quality: Spark Gold jobs perform surrogate key lookups; records that cannot find matching keys are removed or logged, making the Gold layer cleaner than the Silver layer.

3.3. Overview of the Gold Data Warehouse Model

The current Gold model consists of 5 dimension tables and 5 fact tables. This is the central part of the Data Warehouse design chapter.

Dimension Tables

Table	Primary Key	Business Key	Meaning
gold.dim_date	date_key	full_date	Shared time dimension used by fact tables
gold.dim_product	product_key	product_id	Product dimension, supporting SCD Type 2
gold.dim_branch	branch_key	branch_id	Branch and region dimension
gold.dim_department	department_key	department_id	Department/production unit dimension
gold.dim_campaign	campaign_key	campaign_id	Marketing campaign dimension

Fact Tables

Table	Grain	Business Area	Main Measures
gold.fact_sales	1 row = 1 product in 1 order	Sales revenue	quantity, unit_price, unit_cost, revenue, total_cost, profit
gold.fact_marketing_spend	1 row = 1 campaign on	Marketing cost	daily_spend

	1 day in 1 region		
gold.fact_monthly_budget	1 row = 1 region in 1 month	Budget and target	budget_amount, target_revenue, market_size
gold.fact_production_logs	1 row = 1 day + 1 product + 1 department + 1 machine	Production/operations	inventory_level, raw_material_cost, labor_cost
gold.fact_logistics_costs	1 row = 1 branch on 1 day	Logistics	logistics_cost

The dimension tables are placed around the fact tables, while the fact tables are located at the center of the business process flows. Most relationships are one-to-many relationships from dimensions to facts. For example, one row in gold.dim_branch can be linked to many rows in gold.fact_sales and many rows in gold.fact_logistics_costs.

3.4. Design of Dimension Tables

3.4.1. gold.dim_date

gold.dim_date is the time dimension table, shared by multiple fact tables. This table uses a smart key in the YYYYMMDD format as date_key. For example, the date 2026-06-11 has date_key = 20260611. This design makes the time key easy to read, easy to join, and convenient for data checking.

Main attributes:

Column	Meaning
date_key	Primary key in YYYYMMDD format
full_date	Full date
day_of_week	Day of the week
day_of_month	Day of the month

month_number	Month number
month_name	Month name
quarter	Quarter
year	Year

In Spark Gold jobs, dim_date is automatically generated for the date range from 2020-01-01 to 2030-12-31. Creating a predefined calendar table allows all fact tables to join to a consistent time standard, even for dates on which no transactions have occurred yet.

3.4.2. gold.dim_product

gold.dim_product stores product and category information. This is an important dimension because both sales revenue data and production data need to be analyzed by product.

Main attributes:

Column	**Meaning**
product_key	Product surrogate key
product_id	Product ID from the source system
product_name	Product name
category_name	Category name
start_date	Start date of record validity
end_date	End date of record validity
is_current	Flag indicating the current active record

A notable point is that this table is designed according to Slowly Changing Dimension Type 2. The start_date, end_date, and is_current columns allow product attribute changes to be tracked over time. In the current version, the Spark Gold job mainly

loads current product records, but the table structure is already prepared for managing historical changes if product names or categories change in the future.

3.4.3. gold.dim_branch

gold.dim_branch stores branch and regional information. This dimension is used in gold.fact_sales to analyze revenue by branch and in gold.fact_logistics_costs to analyze logistics costs by branch.

Main attributes:

Column	Meaning
branch_key	Branch surrogate key
branch_id	Branch ID from the source system
branch_name	Branch name
region	Region of the branch

In the current model, region is stored as an attribute of dim_branch. This is suitable for the source data because each branch belongs to a specific region. For facts that do not have branch_key, such as marketing spend or monthly budget, region is stored directly in the fact table.

3.4.4. gold.dim_department

gold.dim_department stores information about departments or production units. This dimension is used by gold.fact_production_logs to analyze operational data by department.

Main attributes:

Column	Meaning
department_key	Department surrogate key
department_id	Department ID from the MongoDB source
department_name	Department name

This dimension comes from the MongoDB Production/Operations domain. Bringing the department dimension into the Gold layer allows the dashboard not only to analyze revenue, but also to view raw material costs, labor costs, or inventory levels by operational department.

3.4.5. gold.dim_campaign

gold.dim_campaign stores marketing campaign information. This dimension is used by gold.fact_marketing_spend.

Main attributes:

Column	Meaning
campaign_key	Campaign surrogate key
campaign_id	Campaign ID from the MySQL source
campaign_name	Campaign name
platform	Campaign platform/channel

Separating campaign data into a dimension table helps analyze marketing costs by campaign, platform, and date. This provides the basis for evaluating the effectiveness of programs such as Summer Promotion, Tet Promotion, or Year-End Promotion.

3.5. Design of Fact Tables

3.5.1. gold.fact_sales

gold.fact_sales is the central fact table of the Sales domain. The grain of this table is: one row represents one product in one order. This level of detail is appropriate because one order may contain multiple products, and revenue/profit needs to be analyzed by product.

Main keys and attributes:

Group	Column
Primary key	sales_key

Foreign keys	date_key, product_key, branch_key
Degenerate dimension	order_id
Transaction attributes	sales_channel, customer_segment
Measures	quantity, unit_price, unit_cost, revenue, total_cost, profit

In the Spark job 3_silver_to_gold_sales.py, data from Silver orders is joined with order_details, then the following values are calculated:

revenue = quantity * unit_price

total_cost = quantity * unit_cost

profit = revenue - total_cost

After that, Spark looks up branch_key from gold.dim_branch and product_key from gold.dim_product. Records that cannot find the corresponding surrogate keys are skipped to ensure that the fact table only contains data with valid dimensions. The table has a unique constraint on (order_id, product_key) to support UPSERT operations and prevent duplicate loading when the pipeline is rerun.

3.5.2. gold.fact_marketing_spend

gold.fact_marketing_spend stores marketing costs by campaign, date, and region. The grain of this table is: one row represents one campaign on one day in one region.

Main columns:

Group	Column
Primary key	spend_key
Foreign keys	date_key, campaign_key
Analytical attribute	region
Measure	daily_spend

In the Spark job 3_silver_to_gold_mysql.py, marketing cost data from Silver daily_marketing_spend is converted into date_key, then campaign_key is looked up from gold.dim_campaign. After that, the data is grouped by (date_key, campaign_key,

region) and the total daily_spend is calculated. This table helps the dashboard analyze marketing costs by campaign, date, and region.

3.5.3. gold.fact_monthly_budget

gold.fact_monthly_budget stores budget and target data by month and region. The grain of this table is: one row represents one region in one month.

Main columns:

Group	Column
Primary key	budget_key
Foreign key	date_key
Analytical attribute	region
Measures	budget_amount, target_revenue, market_size

This table is used to compare actual revenue with budget and business targets. The date_key in this table represents the budget month, which is usually generated from budget_month in the Silver layer. Because budget data is stored at the monthly level, when analyzing it in Power BI, it is important not to compare it directly with daily sales facts unless both datasets have been aggregated to the same time granularity.

3.5.4. gold.fact_production_logs

gold.fact_production_logs stores production operation data. The grain of this table is: one row represents one day, one product, one department, and one machine.

Main columns:

Group	Column
Primary key	log_key
Foreign keys	date_key, product_key, department_key
Degenerate dimension	machine_id

Measures	inventory_level, raw_material_cost, labor_cost

In the Spark job 3_silver_to_gold_mongo.py, production log data is used to look up product_key from gold.dim_product and department_key from gold.dim_department. Then, the data is grouped by (date_key, product_key, department_key, machine_id) to aggregate inventory levels, raw material costs, and labor costs. This table extends the analysis from revenue to operations, such as identifying which products have high production costs or which departments generate large costs.

3.5.5. gold.fact_logistics_costs

gold.fact_logistics_costs stores logistics costs by date and branch. The grain of this table is: one row represents one branch on one day.

Main columns:

Group	Column
Primary key	logistics_key
Foreign keys	date_key, branch_key
Measure	logistics_cost

The Spark job 3_silver_to_gold_mongo.py looks up branch_key from gold.dim_branch based on branch_name, then groups the data by (date_key, branch_key) and calculates the total logistics_cost. This table can be combined with fact_sales through date_key and branch_key to evaluate the impact of logistics costs on profit by branch or region.

3.6. Relationships Between Tables in Power BI

The Power BI relationship diagram above shows that the Gold model has been imported into Power BI with one-to-many relationships from dimension tables to fact tables. This is the correct model for a BI dashboard because dimension tables act as filters, while fact tables contain measures for aggregation.

Main relationships:

Dimension	Related Fact	Join Key	Meaning

gold.dim_date	gold.fact_sales	date_key	Analyze revenue over time
gold.dim_date	gold.fact_marketing_spend	date_key	Analyze marketing costs over time
gold.dim_date	gold.fact_monthly_budget	date_key	Analyze budget/target by month
gold.dim_date	gold.fact_production_logs	date_key	Analyze production over time
gold.dim_date	gold.fact_logistics_costs	date_key	Analyze logistics over time
gold.dim_product	gold.fact_sales	product_key	Analyze revenue by product/category
gold.dim_product	gold.fact_production_logs	product_key	Analyze production by product
gold.dim_branch	gold.fact_sales	branch_key	Analyze revenue by branch/region
gold.dim_branch	gold.fact_logistics_costs	branch_key	Analyze logistics by branch
gold.dim_campaign	gold.fact_marketing_spend	campaign_key	Analyze marketing costs by campaign
gold.dim_department	gold.fact_production_logs	department_key	Analyze operations by department

3.7. Keys, Constraints, and UPSERT

The Data Warehouse uses surrogate keys for most dimensions. Keys in the *_key format, such as product_key, branch_key, department_key, and campaign_key, are generated in the Gold layer and used as relationship keys to the fact tables. This approach separates the analytical model from the natural source codes used in OLTP systems.

Business keys are still retained in the dimension tables to support lookups and ensure idempotency:

Dimension	Surrogate Key	Natural Key
gold.dim_product	product_key	product_id
gold.dim_branch	branch_key	branch_id
gold.dim_department	department_key	department_id
gold.dim_campaign	campaign_key	campaign_id
gold.dim_date	date_key	full_date

The file src/init/init_dw.sql also creates unique constraints to support UPSERT operations:

Table	Unique Constraint
gold.dim_branch	branch_id
gold.dim_product	product_id
gold.dim_department	department_id
gold.dim_campaign	campaign_id
gold.fact_sales	order_id, product_key
gold.fact_marketing_spend	date_key, campaign_key, region
gold.fact_monthly_budget	date_key, region
gold.fact_logistics_costs	date_key, branch_key

gold.fact_production_logs	date_key, product_key, department_key, machine_id

The Spark Gold jobs do not write data directly into the main tables immediately. Instead, they first write data into staging tables, then use the INSERT ... ON CONFLICT DO UPDATE statement. This approach allows the pipeline to be executed multiple times without creating duplicate data. This is an important characteristic of a BI pipeline because, during Airflow execution, a task may need to be retried or rerun.

IV. ETL Process

4.1. Overview of the Data Pipeline Architecture

The ETL system in this project is designed based on the Medallion Architecture combined with the Modern Data Stack model. Data flows through separate processing layers, starting from operational sources (OLTP), then moving to the data warehouse layer (OLAP), and finally to the visualization layer (BI). The entire process is automated, independent, and highly reproducible.

The data movement process consists of the following five core layers:

1. Source Systems: The enterprise raw data file.
2. Raw Source Data Layer (Bronze Layer / OLTP): Data is separated from the original file and loaded into three independent database management systems: PostgreSQL, MySQL, and MongoDB. These systems act as simulated operational environments.
3. Intermediate Storage Layer (Data Lake - Medallion Architecture):
    - Bronze Data Lake: Stores historical traces in Parquet file format while preserving the original structure from the source.
    - Silver Data Lake: Contains data that has been cleaned, denoised, error-handled, and standardized by Spark.
4. Integrated Data Warehouse (Gold Layer / Data Warehouse): Clean data from the Silver layer is calculated, matched with keys, and loaded into a centralized PostgreSQL Data Warehouse following the Star Schema model.
5. BI & Analytics Layer: Connects directly to the Gold layer to support intelligent management analysis reports.

4.2. Initializing the Source Systems

Before performing ETL, the project needs to initialize the source systems to simulate an enterprise operational data environment. In reality, business data is usually not stored in a single database, but distributed across multiple systems according to business domains. Therefore, this project uses Docker Compose to simultaneously set up three database management systems: PostgreSQL, MongoDB, and MySQL. These three systems act as the initial source data layer or Bronze layer.

The configuration file used is docker-compose.yml. This file defines the containers, databases, ports, volumes, and network required to run the system locally. Users only need to run a single command in the project root directory to initialize the entire database cluster:

docker compose up -d

The -d parameter allows the containers to run in detached mode. After the command is executed, Docker will download the images if they are not already available on the

machine, create containers, map ports, mount data folders, and automatically run the corresponding database initialization scripts. This is an important step because the later data loading scripts can only run when the source databases are ready.

Services initialized in docker-compose.yml:

Service	Container	Port	Database
PostgreSQL	stg_postgres_sales	5432	sales_db
MongoDB	stg_mongo_production	27017	production_db
MySQL	stg_mysql_finance	3307	finance_db

PostgreSQL - Sales & CRM System

PostgreSQL is used to simulate the sales and CRM system. This is a relational database system suitable for tables with clear structures, primary keys, and foreign keys. In this project, PostgreSQL stores data such as product categories, products, branches, orders, and order details.

Main information:

- Container: stg_postgres_sales
- Database: sales_db
- Host port: 5432
- Initialization script: init-postgres.sql
- Data storage directory: ./data/postgres

The init-postgres.sql script creates source tables including categories, products, branches, orders, and order_details. One notable point is that some columns, such as order_date, are stored as VARCHAR instead of the DATE data type. This design is appropriate for the Bronze layer because the source system needs to accept raw data, including dates with invalid formats. Conversion to a valid date type will be performed in the later ETL steps.

MongoDB - Production & Operations System

MongoDB is used to simulate the production and operations system. This is a NoSQL database that is suitable for data with a more flexible structure than relational tables. In this project, MongoDB stores information about categories, products, departments, production logs, and logistics costs.

Main information:

- Container: stg_mongo_production
- Database: production_db
- Host port: 27017
- Initialization script: init-mongo.js
- Data storage directory: ./data/mongodb

The init-mongo.js script creates collections including Categories, Products, Departments, ProductionLogs, and LogisticsCosts. In addition, the script also inserts several sample department records. Using MongoDB helps the project demonstrate an enterprise scenario in which operational data is stored in a NoSQL system, rather than relying only on relational databases.

MySQL - Finance & Marketing System

MySQL is used to simulate the finance and marketing system. This data group supports the tracking of budgets, revenue targets, marketing costs, and promotion campaigns.

Main information:

- Container: stg_mysql_finance
- Database: finance_db
- Host port: 3307
- Initialization script: init-mysql.sql
- Data storage directory: ./data/mysql

The init-mysql.sql script creates the tables marketing_campaigns, daily_marketing_spend, and monthly_budgets. MySQL inside the container uses the internal port 3306, but it is mapped to host port 3307 to avoid conflicts with any MySQL installation that may already exist on the machine.

Volumes and Network

The three containers are placed in the same data_network, allowing the services to communicate with each other if the pipeline is expanded in the future. Each database is also mounted to a ./data/... directory so that data is not lost when the containers stop. This configuration makes the development process more stable because data is preserved between runs.

Main volumes:

Database	Host Volume	Purpose
PostgreSQL	./data/postgres	Store PostgreSQL data

MongoDB	./data/mongodb	Store MongoDB data
MySQL	./data/mysql	Store MySQL data

After initialization, the container status can be checked using:

docker ps

If all containers are in the running state, the source systems are ready to receive data from the Excel file. If the entire source data needs to be reset and rerun from the beginning, the containers can be stopped and the old data volumes can be deleted. However, the reset operation must be performed carefully because it will remove the data that has already been loaded.

4.3. Loading Data into the Bronze Layer

After the source databases have been initialized, the next step is to load raw data from the Excel file masan_case.xlsx into the Bronze layer. The Bronze layer is the first data layer in the pipeline, responsible for storing data in a state that is as close to the original source as possible. At this layer, the goal is not to fully clean the data, but to move data from the source file into the corresponding operational systems to simulate an enterprise environment.

In this project, data loading is performed by three Python scripts located in the load_data directory:

python load_data/load_postgres.py

python load_data/load_mysql.py

python load_data/load_mongo.py

These three scripts read data from Sheet1 of the masan_case.xlsx file, then select columns suitable for each business domain and write them into the corresponding database. This approach allows data from one initial Excel file to be separated into three independent data sources: Sales/CRM, Production/Operations, and Finance/Marketing.

Overview of data loading scripts:

Script	Target Database	Business Domain	Loaded Tables/Collections

load_postgres.py	PostgreSQL sales_db	Sales & CRM	categories, products, branches, orders, order_details
load_mysql.py	MySQL finance_db	Finance & Marketing	marketing_campaigns, daily_marketing_spend, monthly_budgets
load_mongo.py	MongoDB production_db	Production & Operations	Categories, Products, Departments, ProductionLogs, LogisticsCosts

Loading Sales Data into PostgreSQL

The script load_data/load_postgres.py is responsible for loading sales and CRM data into PostgreSQL. The script starts by reading the Excel file using pandas.read_excel, then standardizes column names by removing leading and trailing spaces.

Main processing steps:

1. Read data from masan_case.xlsx, sheet Sheet1.
2. Extract the list of categories from the Category column and load it into the categories table.
3. Extract the list of products from the Product and Category columns and load it into the products table.
4. Extract the list of branches from the Branch and Region columns and load it into the branches table.
5. Extract order information from OrderID, Date, Branch, Channel, and CustomerSegment, then load it into the orders table.
6. Extract order detail data from OrderID, Product, Quantity, RevenuePerUnit, and UnitCost, then load it into the order_details table.

An important point of this script is that the Date field is converted into a raw string before being loaded into PostgreSQL. This helps the Bronze layer preserve the original data, even when date values are not in a standard format. Numeric columns such as Quantity, RevenuePerUnit, and UnitCost are converted using pd.to_numeric(..., errors='coerce'). If invalid values exist, Pandas converts them to NaN, which is then converted to NULL when loaded into the database.

The script also uses the ON CONFLICT DO NOTHING mechanism when inserting into several tables such as categories, products, branches, orders, and order_details. This approach helps avoid errors when the script is executed multiple times with data that already exists.

Loading Finance and Marketing Data into MySQL

The script load_data/load_mysql.py is responsible for loading finance and marketing data into MySQL. This data group supports the analysis of budgets, targets, marketing costs, and campaign effectiveness.

Main processing steps:

1. Read data from masan_case.xlsx, sheet Sheet1.
2. Extract campaign data from PromotionCampaign and Channel, then load it into the marketing_campaigns table.
3. Extract daily marketing cost data from Date, PromotionCampaign, Region, and MarketingCost, then load it into the daily_marketing_spend table.
4. Create monthly budget data by region from Date, Region, Budget, Target, and MarketSize, then load it into the monthly_budgets table.

In this script, the Date field is also stored as a raw string in the daily marketing cost table. For the monthly budget table, the script attempts to parse Date to create the Month_Year column in the YYYY-MM format. If the date data is invalid, the month value may become NULL. Numeric columns such as MarketingCost, Budget, Target, and MarketSize are converted using pd.to_numeric(..., errors='coerce') to handle invalid data.

The marketing_campaigns table uses ON DUPLICATE KEY UPDATE to avoid errors when a campaign already exists. As a result, the script can be rerun without stopping due to duplicate campaign names.

Loading Production and Operations Data into MongoDB

The script load_data/load_mongo.py is responsible for loading production and operations data into MongoDB. This data is more flexible and includes products, departments, production logs, and logistics costs.

Main processing steps:

1. Read data from masan_case.xlsx, sheet Sheet1.
2. Extract the list of categories from Category and load it into the Categories collection.
3. Extract the list of products from Product and Category, then load it into the Products collection.
4. Extract the list of departments from Department, then load it into the Departments collection.
5. Extract production logs from Date, Product, Department, Stage, Machine, InventoryLevel, RawMaterialCost, and LaborCost, then load them into the ProductionLogs collection.
6. Extract logistics costs from Date, Branch, and LogisticsCost, then load them into the LogisticsCosts collection.

Unlike PostgreSQL and MySQL, MongoDB stores data as documents. Therefore, each record in ProductionLogs or LogisticsCosts is converted into a Python dictionary before being inserted into the collection. The script also deletes old data in the collections using delete_many({}) before inserting new data. This ensures that the loading result is consistent each time the script runs, but it also means that old data in MongoDB will be replaced when the script is rerun.

Similar to the other scripts, the Date field is converted into a raw string, while numeric columns such as InventoryLevel, RawMaterialCost, LaborCost, and LogisticsCost are converted with errors='coerce'. Invalid values are then converted to None and stored as null in MongoDB.

Meaning of the Bronze Layer in the Project

The Bronze layer in this project is not a fully cleaned data layer. It is the layer that stores raw data after it has been loaded into the source systems. Preserving raw data has several important meanings:

- It preserves the data state as close as possible to the original source for traceability.
- It allows the system to accept missing data, incorrectly formatted data, or non-standardized data.
- It separates the data loading stage from the data cleaning stage.
- It accurately simulates real enterprise environments, where operational systems usually prioritize recording generated data first and processing it for analysis later.
- It creates the foundation for later ETL steps to extract, standardize, and load data into the Data Warehouse.

Therefore, warnings related to date values or missing data during Bronze layer loading are not necessarily serious errors. They reflect the characteristics of raw data and will be handled in the later cleaning, standardization, and Data Warehouse loading steps.

4.4. Data Cleaning and Standardization

Instead of processing data sequentially using common data libraries, which may easily lead to out-of-memory (OOM) errors when the data scale increases, the project applies the distributed processing power of Apache Spark (PySpark) combined with the optimized Parquet file storage architecture in the Data Lake layer.

1. Ingesting Raw Data into the Bronze Data Lake

Spark jobs, such as 1a_postgres_to_bronze_dim.py, establish direct JDBC connections to the source database systems. The sole responsibility of this layer is to quickly extract all current data, add an execution-date column, ingest_date, using lit(run_date), and write the data to local storage as high-quality compressed Parquet files. Data in

this layer is kept in its original state, fully preserving any errors inherited from the source systems.

2. Cleaning and Standardizing Data in the Silver Data Lake

At this layer, Spark does not connect to the source databases in order to release source system resources. Instead, Spark reads Parquet files directly from the Bronze layer into distributed memory and performs intensive data processing:

String Cleansing: Spark scans all columns with the String data type and applies trim(col(c_name)) to remove extra spaces. At the same time, dirty string values such as "NaN", "NULL", "N/A", and "none" are converted into actual empty values (None/Null).

Type Casting: Raw string values in quantitative columns are cast into standard data types. Monetary and quantity fields such as quantity, unit_price, and unit_cost are converted to DecimalType(18, 2), while time-related fields are converted using to_date(col("order_date"), "yyyy-MM-dd").

Business Rules Validation: All rows that violate primary key requirements or contain unparseable date values are removed using dropna(subset=[...]). Cases where quantity is negative (quantity < 0) are automatically adjusted to 0.

Deduplication: The dropDuplicates([pk]) function is used based on relational primary keys or composite keys, such as order_id + product_id in the fact table, to ensure uniqueness.

After the data is fully cleaned, it is written to the Silver Data Lake using professional partitioning with .partitionBy("ingest_date").

4.5. Integrating and Loading Data into the Gold Layer (Data Warehouse)

The Gold Layer acts as a complete centralized Data Warehouse. It is configured on an independent PostgreSQL instance named dwh_postgres_gold, running on port 5434, and is completely separated from the source databases.

4.5.1. Data Modeling: Star Schema

Data in the Gold layer is designed according to the Star Schema model, which is optimized for reporting queries. It includes:

Dimension Tables:
dim_date, the central time dimension automatically generated by Spark for the period from 2020 to 2030; dim_product, which applies Slowly Changing Dimension Type 2 to track historical changes in product categories; dim_branch; dim_department; and dim_campaign.

Fact Tables:
fact_sales, with the granularity of one row representing one product in one order; fact_marketing_spend; fact_monthly_budget; fact_production_logs; and fact_logistics_costs.

4.5.2. Safe Data Loading Technology: Staging and Upsert

To load data from the Silver Data Lake into the Gold Data Warehouse, the project implements a staging-based loading technique combined with the psycopg2 connection library.

Staging Step:
Spark reads Parquet files from the Silver layer and performs key-matching operations, such as replacing the natural key branch_id with the surrogate key branch_key through distributed JOIN operations. The processed data is then overwritten using .mode("overwrite") into temporary tables in the Data Warehouse, such as gold.stg_fact_production_logs.

Upsert Step:
An SQL statement using the ON CONFLICT DO UPDATE mechanism, also known as the Upsert Pattern, is executed to move data from the staging table into the official table:

```
INSERT INTO gold.fact_production_logs (
    date_key,
    product_key,
    department_key,
    machine_id,
    inventory_level,
    raw_material_cost,
    labor_cost
)
SELECT
    date_key,
    product_key,
    department_key,
    machine_id,
    inventory_level,
    raw_material_cost,
    labor_cost
FROM gold.stg_fact_production_logs
ON CONFLICT (date_key, product_key, department_key, machine_id)
DO UPDATE SET
    inventory_level = EXCLUDED.inventory_level,
    raw_material_cost = EXCLUDED.raw_material_cost,
```

labor_cost = EXCLUDED.labor_cost;

To ensure that this mechanism does not cause the job to fail, all fact and dimension tables in the Data Warehouse initialization file init_gold_datawarehouse.sql are configured with unique constraints using ALTER TABLE ... ADD CONSTRAINT ... UNIQUE.

4.6. Automatic System Orchestration with Apache Airflow

All separate ETL processes, from Spark jobs to SQL Upsert statements, are connected and centrally managed by Apache Airflow, which acts as the main orchestration system.

The orchestration system is packaged in a workflow architecture file named bi_masan_revenue_pipeline.py, which runs as a DAG (Directed Acyclic Graph). Since the system runs in a local environment, Airflow is configured to use SequentialExecutor together with SQLite metadata in order to minimize RAM usage.

4.6.1. Sequential Design for Resource Optimization

Because Apache Spark tasks consume a large amount of hardware resources, if Airflow triggers all three data streams—Sales, Finance, and Production—to run in parallel, a personal computer may immediately run out of RAM and crash due to OOM.

Therefore, the DAG is optimized to execute the branches sequentially. The Sales data branch runs first and releases memory after completion before handing over execution to the Finance branch and then the Production branch.

The task dependency flow in the DAG is shown below:

[start]
-> [bronze_sales_dim]
-> [bronze_sales_fact]
-> [silver_sales_dim]
-> [silver_sales_fact]
-> [gold_sales]
-> [bronze_mysql_dim]
-> [bronze_mysql_fact]
-> [silver_mysql_dim]
-> [silver_mysql_fact]
-> [gold_mysql]
-> [bronze_mongo_dim]
-> [bronze_mongo_fact]
-> [silver_mongo_dim]

-> [silver_mongo_fact]
-> [gold_mongo]
-> [finish]

4.6.2. Flexible Parameterization Mechanism: Full Load vs Incremental Configuration

The system integrates a flexible execution mode switching mechanism through Jinja2 Template configuration combined with DAG Run Configuration in the Airflow Web UI. The system operator can pass JSON parameters directly when triggering the pipeline.

Parameter for loading the entire history, also known as Full Load:

{ "is_incremental": "False" }

Parameter for loading newly generated data for a specific date, also known as Incremental Load:

{ "is_incremental": "True", "target_date": "2026-06-14" }

Airflow automatically parses this JSON string, converts it into a system environment variable, such as os.getenv("IS_INCREMENTAL"), and passes it directly into the Spark job core to apply the corresponding data filter.

4.7. System Testing and Operational Logs

To ensure data quality and pipeline stability, the ETL testing process is carried out strictly across each layer.

System testing criteria and results:

Test Component	Test Execution Method	Expected Result	Recorded Status
Docker infrastructure	Observe the docker ps command output in the terminal	All 5 system containers are in the Up/Running state	Passed
Source data completeness	Check the total number of rows after Data Seeding	PostgreSQL, MySQL, and MongoDB all match 100% of	Passed

		the row count from the Excel file	
Data Lake integrity	Check the physical directory ./datalake/	All partitioned Parquet structure files appear by date	Passed
Gold layer accuracy	Run a validation query: SELECT COUNT(*) FROM gold.dim_date	The central time dimension is automatically generated with 4,018 rows from 2020 to 2030	Passed
Idempotency	Trigger the DAG continuously 3 times	The system runs successfully, and the row count in the Gold layer remains unchanged without duplicated data	Passed
Foreign key matching	Check rows removed due to mismatched ID codes	Data loss in the MongoDB branch is reduced to 0% after applying the automatic ID generation algorithm	Passed

V. Dashboard Development

5.1. Dashboard Objectives

The dashboard is built using Power BI to visualize data from the Gold Data Warehouse and support users in monitoring the overall business performance. Instead of directly querying data tables in the PostgreSQL Data Warehouse, users can observe key indicators through a visual interface consisting of KPI cards, column charts, combo charts, scatter charts, treemaps, donut charts, and filters by branch, time, and product category.

Based on the dashboard image, the main report page is named **"Business Performance Overview"**. This is an executive overview dashboard page, focusing on important management questions:

- What is the current total revenue?
- What is the total profit?
- What is the overall profit margin of the business?
- How much is the marketing cost?
- How is revenue distributed by product category?
- How do profit and revenue differ across customer segments?
- What is the relationship between ROI and profit?
- Which advertising or sales channel contributes the most profit?
- Which region contributes the highest revenue share?

This dashboard not only presents aggregated figures, but also allows users to filter data across multiple dimensions for deeper analysis. The filters on the left side make the dashboard work as an interactive analytical tool, rather than just a static report image.

5.2. Dashboard Data Source

The dashboard uses data from the project's Gold Data Warehouse. The data has been processed through the Spark pipeline following the Medallion Architecture:

Source Databases
-> Bronze Parquet
-> Silver Parquet
-> Gold Data Warehouse
-> Power BI Dashboard

The direct data source of Power BI is the tables in the gold schema of the PostgreSQL Data Warehouse. In the Power BI interface, the tables are displayed in forms such as gold dim_date, gold fact_sales, gold fact_marketing_spend, and so on, corresponding to the tables in the database.

Group	Tables Used	Role in the Dashboard
Time	gold.dim_date	Filter by year, quarter, month, and day
Product	gold.dim_product	Analyze by product category and product
Branch	gold.dim_branch	Filter by branch and analyze by region
Campaign	gold.dim_campaign	Analyze marketing channels/campaigns
Sales	gold.fact_sales	Calculate revenue, quantity, cost, and profit
Marketing	gold.fact_marketing_spend	Calculate marketing costs
Budget	gold.fact_monthly_budget	Track budget, target, and market size
Logistics	gold.fact_logistics_costs	Track logistics costs
Production	gold.fact_production_logs	Track inventory, raw materials, and labor

In the dashboard shown in the image, the main visuals focus on gold.fact_sales, gold.dim_product, gold.dim_branch, gold.dim_date, and marketing data. These are the core tables used to answer questions about revenue, profit, profit margin, marketing cost, product category, region, and customer segment.

5.3. Overall Dashboard Layout

The dashboard is designed as a one-page overview report. The main background uses a light blue color, while the visuals are placed inside white rounded rectangles. This design helps the charts stand out from the report background and creates a consistent appearance across all components.

The dashboard page can be divided into three main areas:

Area	Components	Purpose
Left column	Dashboard title and slicers	Allows filtering by branch, time, and product category
Top row	4 KPI cards	Quickly displays revenue, profit, profit margin, and marketing cost
Middle and bottom areas	Analytical charts	Analyze by segment, product category, ROI, advertising channel, and region

This design is suitable for a management dashboard because users can read it from top to bottom. First, they can understand the overall KPIs, then look at the charts to understand the causes and contribution structure.

5.4. Interactive Filters

On the left side of the dashboard, there are three main filter groups:

5.4.1. Branch Filter

The Branch filter allows users to select one or more branches to view the entire dashboard within the scope of the selected branch. The values visible in the image include:

- Binh Duong
- Can Tho
- Da Nang
- Other branches in the dataset

When a user selects a specific branch, KPIs such as revenue, profit, profit margin, and marketing cost are recalculated based on that branch. Charts by product category, segment, region, and advertising channel also change accordingly. This filter is very important because branches are direct operating units and are often used to evaluate local business performance.

5.4.2. Time Filter

The Time filter is built from the `full_date` field in the `gold.dim_date` table. In the image, the filter displays the following years:

- 2023
- 2024
- 2025
- 2026

Using a date dimension allows the dashboard to analyze data by year, quarter, month, or day. The time filter helps users view business trends over different periods and compare performance between years.

5.4.3. Product Category Filter

The Product Category filter allows users to select product groups such as:

- Ice cream
- Powdered milk
- Yogurt
- Fresh milk

This filter retrieves data from `gold.dim_product`, specifically from the category field. When a product category is selected, the dashboard shows

how much revenue, profit, and marketing cost that category contributes, as well as how it is distributed by region and channel.

5.5. KPI Cards

The top row of the dashboard contains four KPI cards. This is the most important section because it helps users quickly understand the overall business performance.

KPI	Value in the Image	Meaning
Revenue	13.29M	Total revenue after filters are applied
Profit	2.76M	Total profit from sales activities
Profit Margin	20.80%	Profit ratio over revenue
Marketing Cost	1.08M	Total marketing cost

Each KPI card also includes a small sparkline showing the trend over time. The sparkline helps users not only see the total value, but also understand whether the metric is increasing or decreasing over time. For example, if total revenue is 13.29M but the sparkline declines toward the end of the period, managers may need to analyze the data more deeply by month or by branch.

5.5.1. Revenue

The Revenue KPI is calculated from the `revenue` field in `gold.fact_sales`. This is the most important overall indicator of the dashboard. The value of 13.29M represents the total revenue within the current filter context.

5.5.2. Profit

The Profit KPI is calculated from the `profit` field in `gold.fact_sales`. The value of 2.76M represents the difference between revenue and total sales costs. This metric is more important than revenue when evaluating business performance, because high revenue does not necessarily mean good performance if costs are also high.

5.5.3. Profit Margin

The Profit Margin KPI is calculated by dividing profit by revenue. In the image, the Profit Margin is 20.80%, meaning that every 100 units of revenue generate approximately 20.8 units of profit. This indicator helps evaluate revenue quality and profitability.

5.5.4. Marketing Cost

The Marketing Cost KPI shows the total marketing cost within the current filter context. In the image, the marketing cost is 1.08M. This indicator should be analyzed together with revenue, profit, and ROI to evaluate marketing investment effectiveness.

5.6. Main Analytical Charts

5.6.1. Profit, Revenue, and Marketing ROI by Segment Chart

The large chart in the center of the dashboard is titled **"Average Profit and Average Revenue by Segment"**. This is a combo chart consisting of columns and a line:

- The light blue columns represent profit.
- The dark blue columns represent revenue.
- The orange line represents Marketing ROI.

The segments in the chart include:

- Retail
- E-commerce
- Modern trade channel
- Distributor

This chart helps compare three factors at the same time: revenue, profit, and marketing effectiveness. For example, a segment with high revenue but decreasing Marketing ROI may indicate that marketing costs are increasing faster than the returns generated. Conversely, a segment with stable profit and strong ROI may be a group that should be prioritized for investment.

Analytical meaning:

- Identify customer segments that contribute large revenue.
- Compare profit across segments.
- Evaluate Marketing ROI by segment.
- Detect segments with high revenue but low marketing effectiveness.

5.6.2. Revenue by Category Chart

The horizontal chart on the right is titled **"Revenue by Category"**. This chart analyzes revenue by category or product group.

According to the image, revenue by category includes:

Category	Revenue
Ice cream	5.7M
Yogurt	3.7M
Fresh milk	2.1M
Powdered milk	1.8M

This chart shows that Ice cream is the product group with the highest revenue within the current filter context. Yogurt ranks second, followed by Fresh milk and Powdered milk. This is a very intuitive chart for identifying the revenue structure by product category.

Analytical meaning:

- Identify the product category that contributes the most revenue.
- Compare performance across product groups.
- Support decisions on allocating sales or marketing resources to each category.
- Combine with the branch filter to see which product category performs strongly in each area.

5.6.3. ROI and Profit Scatter Chart

The chart in the lower-left section is titled **"ROI and Profit"**. This is a scatter chart, where the horizontal axis represents ROI in percentage, while the vertical axis represents profit. Each point in the chart represents a data group at the level of detail configured in the visual, such as campaign, channel, product, or segment.

The chart shows that the data points tend to increase from left to right. This suggests that when ROI increases, profit also tends to increase. This visual is suitable for evaluating the relationship between investment effectiveness and profit results.

Analytical meaning:

- Examine the relationship between ROI and profit.
- Detect points with negative ROI or low profit.
- Identify groups with both high ROI and high profit for investment priority.
- Detect outliers, such as high ROI but low profit, or high profit but low ROI.

5.6.4. Treemap of Profit by Advertising Channel

The treemap chart in the lower-middle section is titled **"Profit by Advertising Channel"**. A treemap uses the size of each rectangle to represent the contribution level of each channel. In the image, the displayed channels include:

Channel	Displayed Value
Retail promotion	2.62M
TV	2.42M
Traditional trade	2.12M
Facebook	2.09M
Ecommerce	2.03M
Other	2.01M

The treemap helps users quickly identify which channel contributes the most. Retail promotion is currently the most prominent channel, followed by TV, Traditional trade, Facebook, Ecommerce, and Other.

Analytical meaning:

- Compare profit contribution across channels.
- Identify effective advertising or sales channels.
- Combine with Marketing Cost to evaluate which channels generate good profit with lower cost.
- Support decisions on increasing or reducing marketing budgets by channel.

5.6.5. Donut Chart of Revenue by Region

The donut chart in the lower-right section is titled **"Revenue by Region"**. This chart shows the revenue proportion by region.

According to the image:

Region	Revenue Share
North	41.63%
South	34.91%
Central	23.46%

The North is the region contributing the largest revenue share, accounting for 41.63%. The South ranks second with 34.91%, while the Central region accounts for 23.46%. This chart helps managers quickly understand the geographic revenue structure.

Analytical meaning:

- Identify the main revenue-contributing region.
- Compare contribution levels between regions.
- Combine with the product category filter to see which products perform strongly in each region.
- Combine with logistics costs to evaluate regional efficiency.

VI. Insight Analysis

6.1. Revenue and Profit Overview

According to the overview dashboard, the current business performance is represented by four main KPIs:

Indicator	Value
Revenue	13.29M
Profit	2.76M
Profit Margin	20.80%
Marketing Cost	1.08M

Overall, revenue reached 13.29M and generated 2.76M in profit, corresponding to a Profit Margin of 20.80%. This shows that the business is achieving relatively good profitability: every 100 units of revenue generate approximately 20.8 units of profit. Marketing cost is 1.08M and should continue to be compared with revenue and profit by channel to evaluate investment effectiveness. The sparklines on the KPI cards also show that revenue and profit fluctuate over time, so deeper analysis by segment, product category, and region is needed to identify the causes of increases and decreases.

6.2. Regional Insights

According to the Revenue by Region chart, revenue is distributed across three main regions: North, South, and Central. When filtering each region on the dashboard, the overall KPIs show clear differences between revenue scale and profit efficiency.

Region	Revenue	Profit	Profit Margin	Revenue Share
North	5.53M	1.39M	25.14%	41.63%
South	4.64M	711.53K	15.33%	34.91%

Central	3.12M	662.04K	21.23%	23.46%

The North is the largest revenue-contributing region, accounting for 41.63% of total revenue. In addition to having the highest revenue scale, the North also achieved the highest profit at 1.39M and the highest Profit Margin at 25.14%. This indicates that the North is the most effective business region among the three, as it has both high revenue and a strong profit margin.

The South ranks second in revenue with 4.64M, accounting for 34.91% of total revenue. However, its profit is only 711.53K and its Profit Margin is 15.33%, the lowest among the three regions. This shows that the South has a relatively large market scale, but its profitability is not proportional to its revenue. It is necessary to further analyze product structure, sales costs, logistics costs, or advertising channels in this region to identify the reasons for the lower profit margin.

The Central region has the lowest revenue, reaching 3.12M and accounting for 23.46% of total revenue. However, its Profit Margin is 21.23%, higher than the South. This shows that although the Central region has a smaller scale, its profitability is relatively good. If revenue in this region can be increased while maintaining the current profit margin, the Central region could become a promising growth area.

From the above figures, three main directions can be identified: continue maintaining the strength of the North, optimize costs and sales structure in the South, and seek opportunities to expand revenue in the Central region. In addition, in the dashboard images, the Marketing Cost indicator still displays 1.08M when viewing each region. Therefore, when analyzing marketing effectiveness by region, the data relationships or marketing cost allocation by region should be checked further.

6.3. Product Category Insights

When filtering the dashboard by each product category, the KPIs show clear differences between revenue scale and profitability across product categories.

Product Category	Revenue	Profit	Profit Margin
Ice cream	5.69M	1.19M	20.92%
Yogurt	3.72M	237.32K	6.38%
Fresh milk	2.12M	771.91K	36.43%
Powdered milk	1.76M	564.19K	32.10%

Ice cream is the product category with the highest revenue, reaching 5.69M and generating 1.19M in profit. Its Profit Margin is 20.92%, which is relatively stable. This shows that Ice cream is the key product group in terms of revenue scale and contributes significantly to overall business performance. However, its margin is not the highest, so if the business wants to improve efficiency, it needs to continue controlling sales, marketing, or logistics costs for this category.

Yogurt ranks second in revenue with 3.72M, but its profit is only 237.32K, with a Profit Margin of 6.38%. This is the product category with the lowest profit margin among the four groups. Although Yogurt generates relatively high revenue, its profitability is weak, possibly due to high cost of goods sold, marketing costs, or operating costs. This category should be analyzed more deeply by sales channel and region to identify the causes of reduced profit.

Fresh milk has revenue of 2.12M, lower than Ice cream and Yogurt, but it generates 771.91K in profit with a Profit Margin of 36.43%. This is the product category with the highest profit margin. This shows that although Fresh milk does not have the largest revenue scale, its business efficiency is very strong. Therefore, Fresh milk has expansion potential if the company can increase sales volume while maintaining the current cost structure.

Powdered milk has the lowest revenue among the four groups, reaching 1.76M, but it generates 564.19K in profit and has a Profit Margin of 32.10%. Similar to Fresh milk, Powdered milk has a smaller scale but good profitability. This category should be considered for increased sales or marketing investment if the market still has room for growth.

In summary, Ice cream leads in revenue, while Fresh milk and Powdered milk stand out in profitability. Yogurt is the category that should be prioritized for optimization because it has relatively high revenue but a very low margin. The proposed strategy is to maintain Ice cream revenue, selectively expand Fresh milk and Powdered milk, and review Yogurt costs to improve profitability.

6.4. Channel Insights

When filtering the dashboard by each channel in the Profit by Advertising Channel chart, the KPIs show that each channel has different contribution levels and profitability.

Channel	Revenue	Profit	Profit Margin
Retail promotion	2.62M	922.80K	35.28%

TV	2.42M	318.00K	13.12%
Traditional trade	2.12M	575.18K	27.10%
Facebook	2.09M	145.70K	6.99%
Ecommerce	2.03M	482.90K	23.73%
Other	2.01M	319.98K	15.92%

Retail promotion is the most outstanding channel, with both the highest revenue at 2.62M and the highest profit at 922.80K. Its Profit Margin reaches 35.28%, the highest among all channels. This shows that Retail promotion not only generates strong revenue but also converts revenue into profit very effectively. This channel should be prioritized for maintenance and may be considered for expansion.

Traditional trade has revenue of 2.12M and profit of 575.18K, with a Profit Margin of 27.10%. Although its revenue is not as high as Retail promotion or TV, this channel still has good profitability. It is a stable channel that can continue to be exploited, especially if the company wants to increase profit rather than only increase revenue.

Ecommerce reached revenue of 2.03M, profit of 482.90K, and a Profit Margin of 23.73%. This channel performs fairly well and is aligned with the trend of online shopping. If operating costs, promotions, or order conversion are further optimized, Ecommerce could become an important growth channel.

TV has relatively high revenue, reaching 2.42M, but its profit is only 318.00K and its Profit Margin is 13.12%. This indicates that TV is capable of generating revenue, but its profitability is not proportional to its revenue scale. Media costs, implementation costs, or the product groups sold through this channel should be reviewed to optimize profit.

Other reached revenue of 2.01M, profit of 319.98K, and a Profit Margin of 15.92%. This is an average-performing channel, not particularly outstanding in either revenue or profit. This channel should be analyzed in more detail to identify the components within the “Other” group, avoiding overly general data that reduces decision-making effectiveness.

Facebook is the least effective channel. It generated revenue of 2.09M but profit of only 145.70K, with a Profit Margin of 6.99%. This shows that Facebook may generate revenue, but its costs or conversion effectiveness are not good. This channel should be

prioritized for reviewing campaigns, customer targeting, advertising content, and costs to improve profit.

In summary, Retail promotion is the best channel in terms of both revenue and profit margin. Traditional trade and Ecommerce perform fairly well and should continue to be maintained and optimized. TV and Facebook need tighter cost control because their revenue is not low, but profit is not proportional. In addition, in the dashboard images, Marketing Cost remains at 1.08M when filtering by each channel. Therefore, data relationships or measure calculations should be checked further if marketing cost is to be analyzed accurately by channel.

6.5. Customer Segment Insights

When filtering the dashboard by each customer segment, it can be seen that revenue differences between segments are not too large, but profit and Profit Margin differ quite clearly. This shows that business efficiency by segment does not depend only on revenue scale, but also on costs, product structure, and the ability to convert revenue into profit.

Customer Segment	Revenue	Profit	Profit Margin
Retail	3.41M	919.31K	26.94%
E-commerce	3.38M	735.64K	21.79%
Modern trade channel	3.34M	631.68K	18.94%
Distributor	3.17M	477.93K	15.08%

Retail is the most effective segment. Revenue reached 3.41M, the highest among all segments, while profit reached 919.31K and Profit Margin reached 26.94%. This shows that the Retail segment has both good scale and high profitability. In the Revenue by Category chart, Ice cream and Yogurt are the two major contributing groups in this segment, indicating that Retail may be a suitable channel for promoting popular consumer products.

E-commerce has revenue of 3.38M, nearly equal to Retail, but its profit is lower at 735.64K, with a Profit Margin of 21.79%. This is still a fairly effective segment and is consistent with the online shopping trend. However, the profit gap compared with Retail shows that there is still room to optimize operating costs, promotion costs, or order conversion costs in the online channel.

The Modern trade channel reached revenue of 3.34M, profit of 631.68K, and a Profit Margin of 18.94%. Its revenue is not low, but profitability is at an average level.

Discount policies, display costs, operating costs, or product structure in modern retail systems should be further reviewed to improve margin.

Distributor is the segment with the lowest revenue and profit among the four groups, with revenue of 3.17M, profit of 477.93K, and a Profit Margin of 15.08%. This segment should be closely monitored because it has the lowest profit margin. The causes may come from distributor discount policies, logistics costs, or an unoptimized product structure.

In summary, Retail should be prioritized because it has the best revenue, profit, and profit margin. E-commerce has growth potential but needs cost optimization. The Modern trade channel and Distributor still contribute significant revenue, but their profitability needs improvement, especially Distributor because it has the lowest margin.

6.6. ROI and Profit Insights

The ROI and Profit chart shows the relationship between ROI percentage on the horizontal axis and profit on the vertical axis. The data points in the chart show a clear upward trend from left to right, indicating that higher ROI tends to be associated with higher profit.

In the negative ROI range, from nearly -10% to below 0%, many data points are below the 0 profit level. This means that the business activities or campaigns at these points have not yet generated positive profit. This group should be carefully examined because it may be consuming costs without producing corresponding returns.

In the near-zero ROI range, the data points are concentrated around a profit level close to 0. This group indicates that activities are nearly breaking even and have not generated significant profit. To improve performance, the company needs to optimize costs or increase revenue in this group.

In the positive ROI range, especially from around 5% to 10%, profit increases significantly and can reach nearly 0.1M. This is the most effective group in the chart, showing that channels, segments, or product groups with high ROI often generate better profit.

From this chart, the main insight is that ROI is a good indicator for evaluating profitability. The company should prioritize maintaining and expanding groups with high positive ROI, while reviewing groups with negative ROI to reduce costs, adjust campaigns, or change product/channel structure.

6.7. Business Conclusion

From the overview dashboard, business performance is relatively positive, with revenue of 13.29M, profit of 2.76M, and a Profit Margin of 20.80%. This shows that

the company not only has a significant revenue scale but also maintains fairly good profitability. However, when analyzing by region, product category, channel, and customer segment, it can be seen that high revenue does not always come with high profit efficiency.

In terms of region, the North is the most effective region because it leads in revenue while also having the highest profit and Profit Margin. The South has a large revenue scale but the lowest profit margin, indicating that cost control or sales structure in this region needs to be reviewed. The Central region has lower revenue but a better Profit Margin than the South, so it has growth potential if revenue can be expanded while maintaining current efficiency.

In terms of product category, Ice cream generates the highest revenue and plays the role of a key product group. However, Fresh milk and Powdered milk are the two groups with the highest profit margins. In contrast, Yogurt has fairly high revenue but a low Profit Margin, indicating that cost of goods sold, marketing costs, promotions, or distribution structure should be reviewed for this group.

In terms of sales/advertising channels, Retail promotion is the most outstanding channel because it has high revenue, profit, and Profit Margin. Traditional trade and Ecommerce also perform fairly well. Meanwhile, TV and especially Facebook generate revenue that is not low, but their profit and margin are not proportional, so their costs and conversion effectiveness should be reassessed.

In terms of customer segments, Retail is the best segment because it has the highest revenue and profit. E-commerce has a similar scale but lower profit, indicating that there is still room to optimize operating or conversion costs. The Modern trade channel and Distributor still contribute significant revenue but have lower profit margins, especially Distributor, which should be monitored closely.

The ROI and Profit chart shows a fairly clear positive relationship: negative ROI is often associated with negative or low profit, while high positive ROI is associated with better profit. Therefore, ROI can be used as an important indicator to evaluate the effectiveness of product groups, channels, or segments.

In summary, the BI system shows that the company should not only pursue revenue growth, but also focus on revenue quality and profitability. Groups with high revenue but low margin need cost optimization, while high-margin groups should be considered for expansion to improve overall profit.

6.8. Recommended Actions

Based on the insights above, the following action directions can be proposed:

Maintain and expand the North: The North is currently the most effective region, with high revenue, profit, and Profit Margin. The company should maintain the current strategy in this region and analyze success factors that can be applied to other regions.

Optimize performance in the South: The South has large revenue but low Profit Margin. Logistics costs, discounts, product structure, and sales channel effectiveness in this region should be examined to improve profit.

Pursue selective growth in the Central region: The Central region has a smaller revenue scale but a fairly good margin. Sales or marketing investment can be increased in this region, but it is necessary to ensure that costs do not grow too quickly.

Maintain the Ice cream category: Ice cream is the leading product category in terms of revenue, so sales resources, inventory, and stable distribution should continue to be maintained for this group.

Expand Fresh milk and Powdered milk: These two groups have high Profit Margins and are suitable for selective expansion. The company can try increasing sales budgets or promotional programs to test the ability to increase revenue.

Review the Yogurt category: Yogurt has fairly good revenue but a low margin. Cost of goods sold, promotion costs, distribution costs, or pricing policies should be reviewed to improve profit.

Prioritize Retail promotion: This is the most effective channel in terms of both revenue and profit. The company can continue investing in this channel while analyzing in detail which products and regions are performing well.

Optimize Facebook and TV: These two channels generate revenue that is not low, but profit/margin is not proportional. Advertising costs, campaign content, target customer groups, and conversion rates should be reviewed.

Focus on the Retail segment: Retail is the best-performing segment, so coverage, sales programs, and customer care policies for this group should be maintained.

Improve the Distributor segment: Distributor has the lowest margin among the segments. Discounts, credit policies, logistics costs, and product structure for this group should be reviewed.

Monitor ROI regularly: Groups with negative ROI should be flagged and analyzed for root causes. Groups with high positive ROI should be prioritized for expansion because they are more likely to generate stronger profit.

Complete the cost analysis dashboard: In the current dashboard, Marketing Cost still displays 1.08M when filtering by some dimensions. Relationships or measures should

be checked again to ensure that marketing costs are correctly allocated by region, channel, or segment.

Add target/budget analysis: Additional visuals should be added to compare actual revenue with target and budget by month/region in order to evaluate business plan completion.